

Clase #04 de 27

Caracteres & Strings en C

Abril 28, Martes

Agenda para esta clase

- Caracteres en C
- Introducción a Cadenas en C

Caracteres en C

Caracteres y Alfabetos – Implementación en ANSI C

- Tipo char: *kar* o *char*
- Qué significa: número natural suficientemente grande para guardar un valor ASCII
- Atributos del tipo char
 - Mínimos y máximos: [0,128)
 - Unidad de **sizeof**
 - **assert(1 == sizeof(char));**
 - Tamaño de **char**
 - 1 "byte"
 - Como *mínimo* 8 bits
 - Signado o no signado
 - Diferentes variantes
 - **char**
 - **signed char**
 - **unsigned char**
- Literales Constantes carácter
 - 'A'
 - Muchas otras variantes
 - Diferencias con literales constantes de C++
- ctype.h: Interfaz (i.e., protocolo) de caracteres.

Interfaz ctype.h

- Implementar **ctype.h**.
- Esta vez **toupper** y **tolower** *sin precondiciones*
- Ejemplos
 - **int toupper(int);**
 - **int islower(int);**
- Analizar porqué:
 - ¿**int** y no **char**?
 - ¿**int** y no **bool**?
- Desafíos
 - **Eficiencia:** ¿Cómo hacer más rápido? ¿Máscaras? ¿Tablas?
 - **Internacionalización:** **setlocale**
 - **Modularización:**
 - **ctypetest.c**
 - **ctype.h**
 - **ctype.c**

```
19  /* Hierarchy
20  iscntrl
21  isspace
22  ▶ isblank
23  isprint
24  ▶ isgraph
25  ▶ ▶ ispunct
26  ▶ ▶ isalnum
27  ▶ ▶ ▶ isdigit
28  ▶ ▶ ▶ isxdigit
29  ▶ ▶ ▶ isalpha
30  ▶ ▶ ▶ ▶ isupper
31  ▶ ▶ ▶ ▶ islower
32  ‡ Algunos son cntrl otros print
33  † Son también upper o lower
```

```
1  // Prototypes
2  /// Query Predicates
3  int iscntrl(int);
4  int isspace(int);
5  int isblank(int);
6  int isprint(int);
7  int isgraph(int);
8  int ispunct(int);
9  int isalnum(int);
10 int isdigit(int);
11 int isxdigit(int);
12 int isalpha(int);
13 int isupper(int);
14 int islower(int);
15 /// Transformation
16 int tolower(int);
17 int toupper(int);
```


Ejercicio: To Upper (A Mayúsculas)

- Desarrollar la función `int toupper(int);`
- *Sin precondiciones.*
- ¿Es realmente una operación sobre carácter?

```
28 ▼ static void Test(PunteroAFuncion toupper){  
29     ▶ assert('A'==toupper('a'));  
30     ▶ assert('Z'==toupper('z'));  
31     ▶ assert('@'==toupper('@'));  
32 ▲ }  
33  
34 int toupper0(int c){ if(!islower(c)) return c; return c - ('a'-'A'); }  
35  
36 int toupper1(int c){ if(!islower(c)) return c; return c - 'a' + 'A'; }  
37  
38 int toupper2(int c){ if(!islower(c)) return c; return c - 32; }  
39  
40 int toupper3(int c){ if(!islower(c)) return c; return c - 0x20; }  
41  
42 int toupper4(int c){ if(!islower(c)) return c; return c & 0xDF; }  
43  
44 int toupper5(int c){ if(!islower(c)) return c; return c & 0b1101'1111; }
```

Introducción a Cadenas en C

Formalización, Especificación & Implementación

Implementaciones de Cadenas en C

- Debatar sobre las cadenas en los algunos LP
 - Pascal
 - Turbo Pascal
 - Smalltalk
 - Go
 - Rust
 - JavaScript
 - C++
- Conversar sobre cadenas en C:
 - ¿Son parte del Lenguaje?
 - ¿Son parte de la Biblioteca?
 - ¿Cómo se las representa?
 - ¿En el código fuente como literal (i.e., de forma escrita)?
 - ¿En memoria principal?
 - ¿En memoria secundaria?
 - ¿Tienen tipo de dato?

¿Por qué C no tiene el Tipo String?

- ¿Qué nivel de soporte tiene C para strings?
- No hay una representación directa de strings en los registros, ni hay operaciones en Lenguaje Máquina para manejar strings
- Implicancias de la representación que elige C
 - Facilidad y transparencia
 - Los arreglos no son "*first class citizens*"
 - El tamaño del arreglo no es parte del tipo
 - Contrastar
 - Pascal
 - Java/C#
 - C++
 - Smalltalk
 - Ventajas y desventajas

Tipo de dato de una cadena

- Valores, Variables, y Tipo de Dato
 - ¿ Tipo de Dato Matemático ?
 - a
 - aa
 - aaaaaaa
 - ϵ
 - ¿ Tipo de Dato C?
 - "a"
 - "aa"
 - "aaaaaaa"
 - ""
- ¿Tipo de dato de las cadenas en el Lenguaje Pascal? SQL, C#
- ¿Tipo de dato de las cadenas en el Lenguaje C++?
- ¿Tipo de dato de las cadenas en el Lenguaje C?

Tipo de dato de cualquier cadena

$$\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots \cup \Sigma^{255}$$

$$\bigcup_{i=0}^n \Sigma^i$$

$$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$$

Longitud de una cadena

$$\text{length} : \Sigma^* \rightarrow \mathbb{N} / \text{length}(s) = \begin{cases} 0 & s = \varepsilon \\ 1 + \text{length}(t) & h \cdot t, h \in \Sigma \end{cases}$$

```
13  size_t length(const char *);  
  
28      assert(0==length(""));  
29      assert(1==length("S"));  
30      assert(6==length("Batman"));  
  
33▼ size_t length0(const char *s){  
34      return *s == '\0' ? 0 : 1 + length0(s+1);  
35▲ }  
  
37▼ size_t length1(const char *s){  
38      auto p=s;  
39      while(*p++);  
40      return p-s-1;  
41▲ }
```

Strings en la Biblioteca Estándar:

Interfaces

- `string.h`: principal, maneja la cadena como compuesto, y también sus componentes
 - `strcat`
 - `strlen`
 - `strchr` ...
- `stdio.h`: strings como unidad de intercambio
 - `fputs` `fgets`
 - `fread` `fwrite`
 - `sprintf` `sscanf` ...
- `stdlib.h`: strings como unidad de reserva y transformaciones:
 - `malloc` `calloc`
 - `atoi` `strtod` ...

Preguntas de Repaso

1. ¿Para qué sirve el tipo de dato char?
2. Explique algún atributo del tipo char.
3. ¿Qué provee la interfaz ctype.h?
4. ¿Qué soporte da C para strings?
5. ¿Qué significa Σ^* ?
6. ¿Qué ventajas y desventajas tiene las implementaciones recursiva e iterativas de length?
7. ¿Provee la biblioteca estándar una implementación de length? ¿Conviene usarla?

Términos de la clase

Definir cada término con la bibliografía

- Caracteres
 - Atributos del tipo char
 - Interfaz ctype.h
 - Especificación e implementación de toupper
- String
 - Tipo string en otros LP
 - Strings en C
 - Tipo de dato de un string
 - Tipo de dato de cualquier
 - Definición de Σ^*
 - Implementación recursiva de length
 - Implementación iterativa de length
 - Strings en la Biblioteca Estándar: Interfaces

Referencias Bibliográficas

- K&R
 - 7.8.2 Character Class Testing and Conversion
 - A.4.2 Basic Types
 - B.2 Character Class Tests: <ctype.h>
- MUCH: 1.2 Cadenas
- K&R
 - 1.9 Character Arrays
 - 2.3 Constants
 - 2.4 Declarations
 - 5.3 Pointers and Arrays
 - 7.8.1 String Operations
 - B.1 Input and Output: <stdio.h>
 - B.3 String Functions: <string.h>
 - B.5 Utility Functions: <stdlib.h>

Tareas para la próxima clase

1. Practicar realizando diferentes implementaciones de length
2. Leer el trabajo #1.

¿Consultas?

Fin de la clase